Szyfr Cezara

1. Krótkie wprowadzenie do zajęć – szyfr Cezara

Szyfr Cezara to jeden z najprostszych i najstarszych szyfrów podstawieniowych.
Nazwa pochodzi od Juliusza Cezara, który – według przekazów – miał go używać do tajnej korespondencji wojskowej.

Idea szyfru: przesunięcie liter o stałą liczbę pozycji w alfabecie.

- Ustalamy **klucz** – liczbę k, o ile przesuwamy litery.
- Każdą literę jawnego tekstu zamieniamy na literę położoną **k miejsc dalej w alfabecie**.
- Jeśli „wyskoczymy" poza koniec alfabetu, wracamy na jego początek (działanie modulo długość alfabetu).

Przykład (alfabet łaciński bez polskich znaków, A–Z):

- Klucz $k = 3$
- A → D
- B → E
- X → A (bo po X jest Y, Z, a potem znów A)
- TEKST → WHNVW

W praktyce przyjmujemy uproszczony alfabet (bez ą, ę, ć itd.), a wszystkie litery zamieniamy na **wielkie**.

Matematycznie, jeśli:

- literę zamieniamy na numer: A = 0, B = 1, ..., Z = 25,
- klucz = k,

to:

- **Szyfrowanie:** $C = (P + k) mod 26$
- **Deszyfrowanie:** $P = (C - k) mod 26$

gdzie:

- P– litera tekstu jawnego (plaintext),
- C– litera szyfrogramu (ciphertext).

2. Proste ćwiczenie wprowadzające

Założenia:

- Używamy alfabetu: A, B, C, ..., Z (bez polskich znaków).
- Wszystkie litery zapisujemy wielkimi literami.
- Spacje i znaki interpunkcyjne pomijamy (przepisujemy je bez zmian).

Zadanie

1. Zaszyfruj słowo **KOT** za pomocą szyfru Cezara z kluczem $k = 3$.
2. Zaszyfruj zdanie **ALA MA KOTA** tym samym kluczem $k = 3$.

3. Rozwiązanie krok po kroku

Krok 0. Przygotowanie alfabetu

Zapisujemy alfabet z numeracją (opcjonalnie, ale pomaga):

Litera	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**	**N**	**O**	**P**	**Q**	**R**	**S**	**T**	**U**	**V**	**W**	**X**	**Y**	**Z**
Nr	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Klucz: $k = 3$.

Zadanie 1: KOT, $k = 3$

Tekst jawny: KOT

Rozbijamy na litery: K – O – T

Krok 1: Zamiana liter na numery

Z tabeli:

- K → 10
- O → 14
- T → 19

Krok 2: Dodanie klucza i modulo 26

Dodajemy $k = 3$do każdej liczby i bierzemy resztę z dzielenia przez 26:

1. Dla K:

$$(10 + 3) mod 26 = 13$$

2. Dla O:

$$(14 + 3) mod 26 = 17$$

3. Dla T:

$$(19 + 3) mod 26 = 22$$

Otrzymane numery: 13, 17, 22.

Krok 3: Zamiana numerów na litery

Z tabeli:

- 13 → N

- 17 → R
- 22 → W

Krok 4: Złożenie szyfrogramu

Litery po szyfrowaniu: N – R – W

Szyfrogram: NRW

Zadanie 2: ALA MA KOTA, $k = 3$

Tekst jawny: ALA MA KOTA

Pomijamy spacje, szyfrujemy litery:

ALA MA KOTA → A L A M A K O T A

Krok 1: Zamiana liter na numery

Z tabeli:

- A → 0
- L → 11
- A → 0
- M → 12
- A → 0
- K → 10
- O → 14
- T → 19
- A → 0

Krok 2: Dodanie klucza i modulo 26

Dodajemy $k = 3$:

1. A: $(0 + 3) mod 26 = 3$
2. L: $(11 + 3) mod 26 = 14$
3. A: $(0 + 3) mod 26 = 3$
4. M: $(12 + 3) mod 26 = 15$
5. A: $(0 + 3) mod 26 = 3$
6. K: $(10 + 3) mod 26 = 13$
7. O: $(14 + 3) mod 26 = 17$
8. T: $(19 + 3) mod 26 = 22$
9. A: $(0 + 3) mod 26 = 3$

Otrzymane numery: 3, 14, 3, 15, 3, 13, 17, 22, 3.

Krok 3: Zamiana numerów na litery

- 3 → D
- 14 → O
- 3 → D
- 15 → P
- 3 → D
- 13 → N
- 17 → R
- 22 → W
- 3 → D

Litery: D O D P D N R W D

Krok 4: Złożenie szyfrogramu (z zachowaniem spacji)

Oryginalny układ:
ALA MA KOTA

Szyfrogram:
DOD PD NRW D

Możemy zapisać:

Szyfrogram: DOD PD NRWD
(albo z takimi samymi odstępami jak w oryginale: **DOD PD NRW D**)

Bramki logiczne

1. Kontekst – dlaczego w ogóle bramki?

Komputery pracują na dwóch stanach:

- **0** – brak sygnału / niski poziom napięcia
- **1** – obecność sygnału / wysoki poziom napięcia

Na tych dwóch stanach opiera się **logika Boole'a** – system, w którym wyrażenia mogą być tylko prawdziwe albo fałszywe.
Bramki logiczne to fizyczna realizacja działań na takich wartościach.

Można to porównać do:

- matematyki → gdzie mamy dodawanie, odejmowanie, mnożenie
- logiki cyfrowej → gdzie mamy operacje AND, OR, NOT itd.

Idea dla uczniów:
„Jeśli zrozumiemy, jak działają bramki, zobaczymy, jak w środku myśli komputer."

2. Co to jest bramka logiczna?

Bramka logiczna to prosty układ elektroniczny, który:

- **przyjmuje** sygnały wejściowe (0 lub 1),
- **przetwarza je** zgodnie z określoną regułą logiczną,
- **zwraca** sygnał wyjściowy (0 lub 1).

Każda bramka można opisać:

- **nazwą** (np. AND, OR),
- **symbolem** na schemacie,
- **tabelą prawdy** – czyli zestawieniem wszystkich możliwych wejść i odpowiadających im wyjść.

3. Podstawowe rodzaje bramek

Na zajęciach skupimy się na najważniejszych bramkach:

- **NOT (negacja)** – odwraca sygnał
  - 0 → 1
  - 1 → 0
- **AND (koniunkcja – „i")**
  Wyjście jest 1 tylko wtedy, gdy **wszystkie** wejścia są 1.
- **OR (alternatywa – „lub")**
  Wyjście jest 1, gdy **przynajmniej jedno** wejście jest 1.
- **NAND, NOR** – jak AND/OR, ale z dodatkową negacją na wyjściu

- **XOR (alternatywa wykluczająca)** – wyjście jest 1, jeśli **dokładnie jedno** z wejść jest 1.

Na kolejnych etapach zajęć uczniowie zobaczą, że z tych kilku typów można zbudować dowolny układ cyfrowy.

4. Bramki logiczne w systemach komputerowych

W komputerze setki milionów (a nawet miliardy) bramek logicznych tworzą razem bardziej złożone bloki, takie jak:

- **ALU (Arithmetic Logic Unit)** – jednostka arytmetyczno-logiczna:
    - realizuje działania dodawania, porównywania, operacje AND/OR/XOR na bitach;
- **Rejestry i pamięci** – przechowują bity, które są wynikiem działania bramek;
- **Dekodery i multipleksery** – wybierają, który fragment danych lub która komórka pamięci ma być użyta;
- **Układy sterujące** – decydują, jaki krok wykona procesor w następnej kolejności.

Można uczniom pokazać, że:

- prosty **sumator jednobitowy** (dodający dwa bity) to połączenie bramek AND, OR, XOR,
- całe działania na liczbach w procesorze to tak naprawdę wykonywanie takich operacji bit po bicie.

5. Cele zajęć (propozycja)

Po zakończeniu zajęć uczeń powinien:

1. Wyjaśnić, czym jest bramka logiczna i na czym polega praca w systemie 0/1.
2. Rozpoznać symbole podstawowych bramek (NOT, AND, OR, NAND, NOR, XOR).
3. Odczytać prostą tabelę prawdy i powiązać ją z zachowaniem bramki.
4. Zrozumieć w ogólnym zarysie, jak bramki wykorzystuje się w procesorze i układach cyfrowych.
5. Zbudować (na papierze lub w symulatorze) prosty układ z kilku bramek i przewidzieć jego działanie.

6. Proponowany przebieg wprowadzenia (pierwsze 10–15 min lekcji)

1. **Krótka rozmowa z uczniami:**
    - „Jak komputer zapisuje informacje?” (doprowadzić do pojęcia bitu 0/1).
2. **Wprowadzenie logiki Boole’a:**
    - prawda/fałsz ↔ 1/0.
3. **Definicja bramki logicznej** + pokazanie 2–3 prostych przykładów (NOT, AND, OR).
4. **Pokazanie tabel prawdy** dla tych bramek i wspólne uzupełnienie brakujących wartości.
5. **Powiązanie z komputerem:**

- slajd/rysunek: schematyczna ALU, rejestry – „to wszystko zbudowane jest z bramek”.

Szyfr przestawieniowy macierzowy

1. Wprowadzenie do zajęć – szyfr przestawieniowy macierzowy

Cel lekcji (do wyświetlenia uczniom):

- poznać ideę szyfru przestawieniowego,
- zrozumieć, na czym polega szyfrowanie z użyciem macierzy (tabeli),
- zaszyfrować krótki tekst przy użyciu prostego klucza kolumnowego.

Kontekst:

- Dotychczas poznawaliśmy szyfry, które **zamieniają** litery na inne (np. szyfr Cezara – A → D, B → E, itd.).
- Teraz poznajemy szyfr, który **nie zmienia liter**, tylko **zmienia ich kolejność** – czyli wykonuje **przestawienie** (permutację).

Idea szyfru przestawieniowego macierzowego:

1. Bierzemy tekst jawny (bez polskich znaków i spacji, np. TAJNE DANE → TAJNEDANE).
2. Wpisujemy go **wierszami** do prostokątnej tabeli (macierzy) o ustalonej liczbie kolumn.
3. Nad kolumnami zapisujemy **klucz** – czyli permutację (kolejność, w jakiej będziemy odczytywać kolumny).
4. Szyfrogram powstaje przez **odczytanie liter kolumnami** w kolejności podanej przez klucz.

Przykład na tablicę (krótki, poglądowy):

- Tekst jawny: KOTY
- Liczba kolumn: 2
- Klucz (kolejność kolumn): 2 1

Wpisujemy tekst:

kolumna 1	**kolumna 2**
K	O
T	Y

Klucz: nad kolumną 1 wpisujemy „2”, nad kolumną 2 – „1”:

2	**1**
K	O
T	Y

Odczyt wg klucza:

- najpierw kolumna z numerem **1** → (O, Y) → OY
- potem kolumna z numerem **2** → (K, T) → KT

Szyfrogram: **OYKT**

Uczniowie zwykle szybko łapią ideę: *„te same litery, inna kolejność”*.

2. Proste ćwiczenie wprowadzające (dla uczniów)

Zadanie:

Zaszyfruj wiadomość przy użyciu **szyfru przestawieniowego macierzowego**, korzystając z podanego klucza.

1. Tekst jawny:
   TAJNE DANE
2. Usuń spacje i polskie znaki (tu nie ma polskich znaków):
   TAJNEDANE
3. Ustalono, że używamy macierzy o **4 kolumnach** i następującego **klucza kolumnowego**:

Klucz kolumn: **3 1 4 2**

4. Wpisz tekst jawny do macierzy **wierszami**, po 4 znaki w wierszu.
   Jeśli brakuje liter do wypełnienia ostatniego wiersza, uzupełnij go literą **X**.
5. Zaszyfrogram otrzymasz, odczytując litery **kolumnami** w kolejności rosnących numerów kolumn (najpierw kolumna z numerem 1, potem 2, 3, 4).

Polecenie dla uczniów (do zapisania):

Zastosuj szyfr przestawieniowy macierzowy z 4 kolumnami i kluczem
3 1 4 2, aby zaszyfrować tekst TAJNE DANE.
Pamiętaj o usunięciu spacji, uzupełnieniu ostatniego wiersza literą X
i odczytaniu kolumn w kolejności: 1, 2, 3, 4.
Zapisz otrzymany szyfrogram.

3. Rozwiązanie krok po kroku (dla nauczyciela)

Krok 1. Przygotowanie tekstu jawnego

Tekst:
TAJNE DANE

Usuwamy spacje:
TAJNEDANE

Długość tekstu: 9 znaków.

Krok 2. Dobór rozmiaru macierzy i dopełnienie

Liczba kolumn: **4**
Potrzebujemy tylu pól w macierzy, aby pomieścić cały tekst.
Dzielimy długość tekstu przez liczbę kolumn:

- 9 znaków / 4 kolumny → potrzebne są **3 wiersze** (bo 2 wiersze dają tylko 8 pól).

Liczba pól w macierzy:

- 3 wiersze × 4 kolumny = 12 pól

Brakuje nam:

- 12 − 9 = **3 znaki**

Dopełniamy tekst trzema literami **X**:

TAJNEDANE + XXX = **TAJNEDANEXXX**

Krok 3. Wpisanie tekstu do macierzy

Wpisujemy tekst **wierszami**, po 4 znaki w wierszu:

- 1. wiersz: TAJN
- 2. wiersz: EDAN
- 3. wiersz: EXXX

Macierz:

	kol. 1	**kol. 2**	**kol. 3**	**kol. 4**
w1	T	A	J	N
w2	E	D	A	N
w3	E	X	X	X

Krok 4. Zapisanie klucza nad kolumnami

Klucz: **3 1 4 2**

To znaczy, że:

- nad **kolumną 1** wpisujemy **3**
- nad **kolumną 2** – **1**
- nad **kolumną 3** – **4**
- nad **kolumną 4** – **2**

Tabela z kluczem:

numer kolumny w kluczu	**3 1 4 2**
kolumna	1 2 3 4
w1	T A J N
w2	E D A N
w3	E X X X

Krok 5. Odczytywanie szyfrogramu według klucza

Odczytujemy **kolumnami**, w kolejności numerów **1, 2, 3, 4**.

1. **Kolumna z numerem 1 w kluczu**
   Numer 1 jest nad **kolumną 2** (druga kolumna).
   Odczytujemy ją z góry na dół:
   - A (w1)
   - D (w2)
   - X (w3)

Daje to fragment szyfrogramu: **ADX**

2. **Kolumna z numerem 2 w kluczu**
   Numer 2 jest nad **kolumną 4** (czwarta kolumna):
   - N (w1)
   - N (w2)
   - X (w3)

Dopisujemy: **NNX**
Szyfrogram do tej pory: **ADXNNX**

3. **Kolumna z numerem 3 w kluczu**
   Numer 3 jest nad **kolumną 1** (pierwsza kolumna):
   - T (w1)
   - E (w2)
   - E (w3)

Dopisujemy: **TEE**

Szyfrogram: **ADXNNXTEE**

4. **Kolumna z numerem 4 w kluczu**
   Numer 4 jest nad **kolumną 3** (trzecia kolumna):
   - J (w1)
   - A (w2)
   - X (w3)

Dopisujemy: **JAX**
Ostateczny szyfrogram: **ADXNNXTEEJAX**

Odpowiedź:

Szyfrogram: ADXNNXTEEJAX

Systemy liczbowe

1. Systemy liczbowe – po co nam one w informatyce?

W życiu codziennym używamy **systemu dziesiętnego** (podstawa 10, cyfry 0–9).
Komputer natomiast „myśli" inaczej – jego podstawą jest **system binarny** (podstawa 2, cyfry 0 i 1).

1.1. Co to jest system liczbowy?

- **Podstawa systemu** – liczba różnych cyfr, które można użyć (np. 10 w dziesiętnym, 2 w binarnym).
- **Cyfry** – symbole używane do zapisu liczby:
  - dziesiętny: 0–9
  - binarny: 0, 1
  - ósemkowy: 0–7
  - szesnastkowy: 0–9, A, B, C, D, E, F
- **Pozycja cyfry** decyduje o jej wartości (system pozycyjny).
  Przykład (dziesiętnie):

$$472_{10} = 4 \cdot 10^2 + 7 \cdot 10^1 + 2 \cdot 10^0$$

Analogicznie w binarnym:

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11_{10}$$

1.2. System binarny w komputerze

Komputer opiera się na dwóch stanach:

- **0 / 1**
- brak napięcia / jest napięcie
- fałsz / prawda

Dlatego:

- dane, programy, obrazy, dźwięki – wszystko jest ostatecznie zapisane jako **ciągi bitów** (0 i 1),
- **bit** – najmniejsza jednostka informacji (0 lub 1),
- **bajt** – 8 bitów (np. 01001101_2).

1.3. Inne systemy ważne w informatyce

- **System ósemkowy (oktalny)** – wygodny skrót zapisu binarnego (3 bity = 1 cyfra ósemkowa).
- **System szesnastkowy (heksadecymalny)** – 4 bity = 1 cyfra szesnastkowa.
  Przykłady użycia:
  - adresy pamięci (np. 0x7FFDE3A0),

- kolory w HTML/CSS (#FF0000 – czerwony),
- kody maszynowe, instrukcje procesora,
- zapisy kluczy kryptograficznych.

Dla uczniów warto pokazać proste zamiany:

- z dziesiętnego na binarny,
- z binarnego na szesnastkowy (grupowanie po 4 bity).

2. Zastosowanie systemów liczbowych w systemach komputerowych

Na lekcji możesz podkreślić, że systemy liczbowe to nie „sucha teoria", ale podstawa działania komputerów:

1. **Reprezentacja danych**
   - liczby całkowite, zmiennoprzecinkowe, znaki (ASCII, Unicode),
   - każdy znak → kod liczbowy → zapis w postaci bitów.
2. **Adresowanie pamięci**
   - każda komórka pamięci ma swój adres (często podawany szesnastkowo),
   - procesor posługuje się adresami, aby odczytać/zapisać dane.
3. **Instrukcje procesora**
   - program w wysokim języku (C, Java itd.) jest tłumaczony na **kod maszynowy** – ciąg binarny,
   - heksadecymalny zapis jest tylko „bardziej przyjazną" formą dla człowieka.
4. **Kryptografia**
   - klucze, szyfrogramy, hashe – to po prostu duże liczby zapisane w różnych systemach,
   - często prezentowane w postaci szesnastkowej (np. 3A5F09C4...).

To naturalnie prowadzi do tematu szyfrowania – i tutaj wchodzi DES.

Zasada działania DES (Data Encryption Standard)

DES (Data Encryption Standard) to klasyczny algorytm szyfrowania blokowego, opracowany w latach 70.
Dziś nie jest już uważany za bezpieczny, ale jest świetnym przykładem do nauki podstaw kryptografii.

3.1. Podstawowe parametry DES

- **Rodzaj algorytmu:** symetryczny (ten sam klucz do szyfrowania i deszyfrowania),
- **Szyfr blokowy:** przetwarza dane w blokach:
    - rozmiar bloku: **64 bity**,
- **Klucz:** nominalnie 64 bity, ale:
    - 8 bitów to bity parzystości,
    - realnie siła klucza: **56 bitów**.

3.2. Struktura – sieć Feistela

DES jest przykładem tzw. **sieci Feistela**:

1. Dane wejściowe (64 bity) są dzielone na dwie części:
    - lewą: **L** (32 bity),
    - prawą: **R** (32 bity).
2. Algorytm wykonuje **16 rund** tego samego rodzaju operacji.
3. W każdej rundzie:
    - z klucza głównego generowany jest **podklucz rundy** (48 bitów),
    - na prawej części wykonywana jest funkcja f z użyciem podklucza,
    - wynik jest łączony (XOR) z lewą częścią.

Schemat pojedynczej rundy (upraszczając):

- Wejście: L_{i-1}, R_{i-1}
- Obliczenia:
    - $L_i = R_{i-1}$
    - $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$

gdzie:

- K_i– podklucz i-tej rundy,
- $\oplus$– operacja XOR (modulo 2).

3.3. Funkcja rundowa f – „serce” DES

Funkcja **f** robi kilka rzeczy:

1. **Rozszerzenie (E)**
   32-bitowe R jest rozszerzane do **48 bitów** (część bitów powtarzana) według ustalonej permutacji.
2. **Mieszanie z podkluczem**
   Rozszerzone R jest łączone z podkluczem K_i za pomocą XOR:

$$R' = E(R) \oplus K_i$$

3. **S-boxy (tablice podstawieniowe)**
   48-bitowy wynik dzieli się na 8 grup po 6 bitów.
   Każda grupa przechodzi przez swój **S-box** → z 6 bitów robi 4 bity.
   W ten sposób z 48 bitów powstaje **32-bitowy** wynik.
   To S-boxy wprowadzają nieliniowość i „zamieszanie” w szyfrze.
4. **Permutacja P**
   Końcowa permutacja porządkuje te 32 bity według ustalonego schematu.

Ten 32-bitowy wynik to właśnie f(R_{i-1}, K_i) używane w równaniu rundy.

3.4. Permutacje początkowa i końcowa

Dodatkowo DES stosuje:

- **IP (Initial Permutation)** – permutacja początkowa 64 bitów bloku,
- po 16 rundach – zamianę miejscami L i R,
- **FP (Final Permutation)** – permutację końcową (odwrotność IP).

Te permutacje nie dodają bezpieczeństwa, ale są częścią standardu.

3.5. Deszyfrowanie

Dzięki strukturze Feistela:

- deszyfrowanie polega na **wykonaniu tych samych rund w odwrotnej kolejności kluczy** (zamiast $K_1, K_2, \ldots, K_{16}$używamy $K_{16}, K_{15}, \ldots, K_1$),
- ta sama implementacja może obsłużyć i szyfrowanie, i deszyfrowanie.

4. Jak to związać na zajęciach?

Propozycja uporządkowania wprowadzenia:

1. **Systemy liczbowe**
    - przypomnienie dziesiętnego,
    - wyjaśnienie binarnego, ósemkowego, szesnastkowego,
    - proste przykłady konwersji.
2. **Zastosowanie w komputerach**
    - bity, bajty, kodowanie znaków,
    - adresy pamięci, kolory, instrukcje maszynowe.
3. **Przejście do kryptografii**
    - informacja = liczby = bity,
    - szyfrowanie = wykonywanie określonych operacji na liczbach/binarnych blokach.
4. **DES jako przykład algorytmu szyfrującego**
    - parametry (blok, klucz),
    - 16 rund, sieć Feistela,
    - funkcja rundowa f, S-boxy,
    - ta sama struktura dla szyfrowania i deszyfrowania.

Ćwiczenie: DES – pytania i prosta runda szyfrowania

Zadanie 1 – pytania teoretyczne (rozgrzewka)

1. Podaj podstawowe parametry algorytmu DES:
   a) rozmiar bloku danych (w bitach)
   b) długość klucza (w bitach – rzeczywista „siła" klucza)
   c) liczbę rund
   d) czy DES jest algorytmem symetrycznym czy asymetrycznym?

Zadanie 2 – uproszczona jedna runda DES (wersja „mini")

W tym zadaniu używamy **uproszczonej rundy DES** (tzw. sieć Feistela), aby przećwiczyć:

- dzielenie bloku na lewą i prawą część,
- operację XOR,
- działanie uproszczonej funkcji rundowej f.

Mamy 8-bitowy blok danych (dla uproszczenia, normalnie DES ma 64 bity):

- Lewa część początkowa:

$$L_0 = 1010\ 1100$$

- Prawa część początkowa:

$$R_0 = 0110\ 0011$$

Oraz 8-bitowy klucz rundowy:

- $K_1 = 1111\ 0000$

Uproszczona runda działa tak:

1. **Funkcja f(R_0, K_1)**
   a) oblicz:

$$X = R_0 \oplus K_1$$

b) powstały wynik X podziel na dwie połowy po 4 bity (lewy i prawy „półbajt"),
c) do **każdego** 4-bitowego fragmentu zastosuj poniższą „mini S-box" (podstawienie):

Wejście (4 bity)	**Wyjście (4 bity)**
0000	1010
0001	1011
0010	1000
0011	1001
0100	1110
0101	1111
0110	1100
0111	1101
1000	0010
1001	0011
1010	0000
1011	0001
1100	0110
1101	0111
1110	0100

Wejście (4 bity)	Wyjście (4 bity)
1111	0101

(To jest **wymyślona, uproszczona tablica**, tylko do ćwiczeń – nieprawdziwy S-box DES).

d) sklej z powrotem oba 4-bitowe wyniki w jeden ciąg 8-bitowy: to jest

$$f(R_0, K_1)$$

2. **Aktualizacja L i R (schemat Feistela):**

$$\begin{cases} L_1 = R_0 \\ R_1 = L_0 \oplus f(R_0, K_1) \end{cases}$$

Zadanie:

1. Oblicz krok po kroku:
   - $X = R_0 \oplus K_1$
   - $f(R_0, K_1)$
2. Na tej podstawie znajdź:
   - L_1
   - R_1

Rozwiązanie krok po kroku

Rozwiązanie zadania 1

1. Podstawowe parametry DES:

a) **Rozmiar bloku danych:**
DES szyfruje dane w blokach po **64 bity**.

b) **Długość klucza:**
Klucz ma nominalnie 64 bity, ale co 8. bit jest bitem parzystości, więc realna „siła" klucza to **56 bitów**.

c) **Liczba rund:**
DES wykonuje **16 rund**.

d) **Typ algorytmu:**
DES jest algorytmem **symetrycznym** (do szyfrowania i deszyfrowania używa się tego samego klucza – tylko w deszyfrowaniu podklucze stosuje się w odwrotnej kolejności).

Rozwiązanie zadania 2 – runda DES (uproszczona)

Dane początkowe

- $L_0 = 1010\ 1100$
- $R_0 = 0110\ 0011$

- $K_1 = 1111\ 0000$

Krok 1: Obliczamy $X = R_0 \oplus K_1$

Zapisujemy liczby jedna pod drugą:

R_0 = 0\ 1\ 1\ 0\ \ 0\ 0\ 1\ 1 \\ K_1 = 1\ 1\ 1\ 1\ \ 0\ 0\ 0\ 0 \\ \hline X = ?\ ?\ ?\ ?\ \ ?\ ?\ ?\ ?

Wykonujemy XOR bit po bicie (pamiętaj: 0⊕0=0, 1⊕1=0, 0⊕1=1, 1⊕0=1):

1. $0 \oplus 1 = 1$
2. $1 \oplus 1 = 0$
3. $1 \oplus 1 = 0$
4. $0 \oplus 1 = 1$
5. $0 \oplus 0 = 0$
6. $0 \oplus 0 = 0$
7. $1 \oplus 0 = 1$
8. $1 \oplus 0 = 1$

Zatem:

$$X = 1\ 0\ 0\ 1\ 0\ 0\ 1\ 1 = 1001\ 0011$$

Krok 2: Dzielenie X na dwie połowy (po 4 bity)

$$X = 1001\ 0011$$

- lewa połowa (4 bity): **1001**
- prawa połowa (4 bity): **0011**

Zapisujemy:

$$\text{Lewy nibble} = 1001 \text{Prawy nibble} = 0011$$

Krok 3: Zastosowanie mini S-boxa

Korzystamy z tabeli:

- Dla **1001** szukamy w tabeli:
  1001 → **0011**
- Dla **0011**:
  0011 → **1001**

Zatem po przejściu przez S-box:

- lewy wynik: **0011**
- prawy wynik: **1001**

Łączymy z powrotem:

$$f(R_0, K_1) = 0011\ 1001$$

Krok 4: Obliczamy L_1 i R_1

Z definicji rundy Feistela:

$$\begin{cases} L_1 = R_0 \\ R_1 = L_0 \oplus f(R_0, K_1) \end{cases}$$

4.1. Lewa część po rundzie

$$L_1 = R_0 = 0110\ 0011$$

Gotowe.

4.2. Prawa część po rundzie: $R_1 = L_0 \oplus f(R_0, K_1)$

Przepisujemy obie liczby:

L_0 = 1\ 0\ 1\ 0\ \ 1\ 1\ 0\ 0 \\ f(R_0, K_1) = 0\ 0\ 1\ 1\ \ 1\ 0\ 0\ 1 \\ \hline R_1 = ?\ ?\ ?\ ?\ \ ?\ ?\ ?\ ?

XOR bit po bicie:

1. $1 \oplus 0 = 1$
2. $0 \oplus 0 = 0$
3. $1 \oplus 1 = 0$
4. $0 \oplus 1 = 1$
5. $1 \oplus 1 = 0$
6. $1 \oplus 0 = 1$
7. $0 \oplus 0 = 0$
8. $0 \oplus 1 = 1$

Dostajemy:

$$R_1 = 1\ 0\ 0\ 1\ 0\ 1\ 0\ 1 = 1001\ 0101$$

Odpowiedź końcowa dla zadania 2

- $X = R_0 \oplus K_1 = 1001\ 0011$
- $f(R_0, K_1) = 0011\ 1001$
- $L_1 = 0110\ 0011$
- $R_1 = 1001\ 0101$

Możesz jeszcze poprosić uczniów, żeby zapisali wynikowy blok po 1 rundzie jako całość:

$$\text{blok po 1 rundzie} = L_1 \parallel R_1 = 0110\ 0011\ 1001\ 0101$$

Metoda płotkowa

1. Wprowadzenie do zajęć – metoda płotkowa

Co to jest metoda płotkowa?

Metoda płotkowa (ang. *rail fence cipher*) to **szyfr przestawieniowy**, w którym:

- nie zamieniamy liter na inne (jak w szyfrze Cezara),
- tylko **zmieniamy kolejność liter** w tekście jawnym.

Nazwa „płotkowa" pochodzi od sposobu zapisu tekstu:

- litery wpisujemy „zygzakiem" w kilku wierszach,
- a potem odczytujemy je wierszami – jakby z desek płotu.

Podstawowa idea

1. **Wybieramy klucz** – jest nim **liczba wierszy** (np. 2, 3, 4).
2. **Zapisujemy tekst zygzakiem**:
    - zaczynamy w pierwszym wierszu,
    - schodzimy w dół wiersz po wierszu,
    - po osiągnięciu ostatniego wiersza wracamy w górę, i tak w kółko.
3. **Szyfrogram powstaje** przez odczytanie liter:
    - najpierw z pierwszego wiersza,
    - potem z drugiego,
    - ... aż do ostatniego.

To jest szyfr **tylko przestawieniowy** – statystyka liter pozostaje taka sama, zmienia się układ tekstu.

Przykładowy zapis zygzakiem (3 wiersze)

Tekst jawny (bez spacji):
KRYPTOGRAFIA

Dla 3 wierszy układamy litery zygzakiem:

- Wiersz 1: K T A A
- Wiersz 2: R Y O R F
- Wiersz 3: P G I

(pełny zapis pokażę niżej w rozwiązaniu zadania)

Cele zajęć

Uczeń po zajęciach powinien umieć:

- wyjaśnić, że metoda płotkowa to szyfr przestawieniowy,
- zaszyfrować prosty tekst metodą płotkową dla danego klucza (liczby wierszy),
- rozszyfrować krótki szyfrogram, znając klucz,
- narysować „płotek” (układ zygzakowy) i poprawnie wpisać litery.

2. Proste ćwiczenie wprowadzające

Zadanie

Zaszyfruj metodą płotkową (3 wiersze) następujący tekst:

Tekst jawny:
TAJNE DANE

a) usuń spacje,
b) zastosuj 3 wiersze (klucz = 3),
c) podaj otrzymany szyfrogram.

3. Rozwiązanie krok po kroku

Krok 1. Przygotowanie tekstu jawnego

Tekst:
TAJNE DANE

Usuwamy spacje (zwykle w prostych ćwiczeniach pomijamy także polskie znaki, tu ich nie ma):

Tekst roboczy:
TAJNEDANE

Dla czytelności ponumerujmy litery:

1. T
2. A
3. J
4. N
5. E
6. D
7. A
8. N
9. E

Krok 2. Ustalenie klucza

Klucz = liczba wierszy = **3**.

Będziemy więc rysować **3 wiersze** i wpisywać litery zygzakiem:

- w dół (wiersz 1 → 2 → 3),
- w górę (wiersz 3 → 2 → 1),
- i tak dalej.

Krok 3. Zapis tekstu zygzakiem

Tworzymy 3 wiersze (na razie puste):

- Wiersz 1:
- Wiersz 2:
- Wiersz 3:

Teraz wpisujemy litery po kolei, poruszając się zygzakiem.

Ruch 1 – w dół

1. Litera **T** → wiersz 1
    - W1: T
    - W2:
    - W3:
2. Litera **A** → wiersz 2
    - W1: T
    - W2: A
    - W3:
3. Litera **J** → wiersz 3
    - W1: T
    - W2: A
    - W3: J

Jesteśmy na ostatnim wierszu (3), teraz będziemy iść **w górę**.

Ruch 2 – w górę

4. Litera **N** → wiersz 2
    - W1: T
    - W2: A N
    - W3: J
5. Litera **E** → wiersz 1
    - W1: T E
    - W2: A N
    - W3: J

Jesteśmy znów na pierwszym wierszu, więc znowu **idziemy w dół**.

Ruch 3 – znów w dół

6. Litera **D** → wiersz 2
    - W1: T E
    - W2: A N D
    - W3: J
7. Litera **A** → wiersz 3
    - W1: T E
    - W2: A N D
    - W3: J A

Znowu jesteśmy na ostatnim (3), więc **wracamy w górę**.

Ruch 4 – w górę

8. Litera **N** → wiersz 2
    - W1: T E
    - W2: A N D N
    - W3: J A
9. Litera **E** → wiersz 1
    - W1: T E E
    - W2: A N D N
    - W3: J A

Koniec liter – zapełniony płotek wygląda tak:

- **Wiersz 1:** T E E
- **Wiersz 2:** A N D N
- **Wiersz 3:** J A

Dla lepszego wyobrażenia można też zapisać w formie „schodków” (niekonieczne, ale pomocne na tablicy):

W1: T E E

W2: A N D N

W3: J A

Krok 4. Odczytanie szyfrogramu

Szyfrogram to odczytanie liter **wierszami od góry do dołu**:

1. Najpierw cały wiersz 1:
   TEE
2. Potem cały wiersz 2:
   ANDN
3. Na końcu wiersz 3:
   JA

Łączymy wszystko w jeden ciąg:

Szyfrogram:
TEEANDNJA

Można go zapisać:

- bez przerw: TEEANDNJA,
- lub dla czytelności w grupach (np. po 3): TEE AND NJA – ale to już kwestia formatu, nie samego szyfru.

4. Propozycja krótkiego podsumowania na koniec zajęć

Na zakończenie zajęć można poprosić uczniów, aby:

1. Sami wymyślili krótki wyraz (8–10 liter),
2. Zaszyfrowali go metodą płotkową dla 2 i dla 3 wierszy,
3. Zwrócili uwagę, że **ten sam tekst jawny** dla różnych kluczy daje **różne szyfrogramy**.

Szyfr Vigenère’a

1. Wprowadzenie do szyfru Vigenère’a

1.1. Idea ogólna

Szyfr Vigenère’a to:

- **szyfr podstawieniowy polialfabetyczny** – każda litera tekstu jawnego może być szyfrowana inną „alfabetową przesuniętą wersją”,
- wykorzystuje **słowo-klucz**, które decyduje, o ile miejsc przesuwamy litery.

W przeciwieństwie do szyfru Cezara (jedno stałe przesunięcie dla całego tekstu), w szyfrze Vigenère’a:

- każde kolejne przesunięcie zależy od odpowiedniej litery klucza,
- klucz powtarza się cyklicznie nad tekstem.

Dzięki temu szyfr:

- jest trudniejszy do złamania metodami analizy częstotliwości,
- był długo uważany za „niezłamany” (nazywano go „szyfrem nie do złamania”).

1.2. Alfabet i reprezentacja liter

Dla uproszczenia przyjmujemy:

- tylko litery **A–Z** (bez polskich znaków),
- brak spacji i znaków interpunkcyjnych (usuwamy lub ignorujemy),
- litery zapisujemy **wielkimi literami**.

Każdej literze przypisujemy numer:

Litera	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**
Numer	0	1	2	3	4	5	6	7	8	9	10	11	12

Litera	**N**	**O**	**P**	**Q**	**R**	**S**	**T**	**U**	**V**	**W**	**X**	**Y**	**Z**
Numer	13	14	15	16	17	18	19	20	21	22	23	24	25

1.3. Zasada działania

Mamy:

- **tekst jawny** (plaintext),
- **klucz** (key),
- **tekst zaszyfrowany** (ciphertext, kryptogram).

1. Piszesz tekst jawny.
2. Pod tekstem powtarzasz słowo-klucz tyle razy, aby długość się zgadzała.
3. Każdą literę szyfrujesz, przesuwając ją o tyle pozycji w alfabecie, jaki numer ma litera klucza na danej pozycji.

Wzór (dla liter zamienionych na numery):

- Szyfrowanie:

$$C_i = (P_i + K_i) \mathrm{mod}\ 26$$

- Deszyfrowanie:

$$P_i = (C_i - K_i) \mathrm{mod}\ 26$$

gdzie:

- P_i – numer i-tej litery tekstu jawnego,
- K_i – numer i-tej litery klucza,
- C_i – numer i-tej litery szyfrogramu.

1.4. Tablica Vigenère'a (w skrócie)

Często używa się **tablicy Vigenère'a**:

- Wiersze: alfabet przesunięty o 0, 1, 2, ... 25 pozycji.
- Kolumny: litery tekstu jawnego.
- Wybór odpowiedniej litery:
    - wybierz wiersz odpowiadający literze klucza,
    - wybierz kolumnę odpowiadającą literze tekstu jawnego,
    - przecięcie wiersza i kolumny = litera szyfrogramu.

Na zajęciach możesz:

- pokazać wydruk tablicy,
- kazać uczniom np. odnaleźć zaszyfrowaną literę dla danych: litera jawna + litera klucza.

1.5. Zalety i wady szyfru Vigenère'a (krótko)

Zalety:

- trudniejszy do złamania niż prosty szyfr Cezara,
- rozkład częstotliwości w szyfrogramie jest „wygładzony".

Wady:

- jeśli klucz jest krótki i powtarzalny, można stosować metody kryptoanalizy (np. Kasiski, analiza okresu klucza),
- nadal podatny na ataki przy dłuższych tekstach.

2. Proste ćwiczenie wprowadzające

Treść ćwiczenia

1. Zaszyfruj metodą Vigenère'a tekst jawny:

TEKST JAWNY:
KOMPUTER

używając klucza:

KLUCZ:
SZYFR

2. Następnie **odszyfruj** otrzymany szyfrogram, używając tego samego klucza, aby sprawdzić poprawność.

3. Rozwiązanie krok po kroku

3.1. Przygotowanie danych

Tekst jawny:
K O M P U T E R

Klucz: SZYFR

Powtarzamy klucz, aby miał długość 8 (jak tekst jawny):

- K O M P U T E R
- S Z Y F R S Z Y ← powtarzający się klucz

Czyli:

Pozycja	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Tekst jawny	K	O	M	P	U	T	E	R
Klucz	S	Z	Y	F	R	S	Z	Y

3.2. Zamiana liter na numery

Z tabeli alfabetu:

- K = 10
- O = 14
- M = 12
- P = 15
- U = 20
- T = 19
- E = 4
- R = 17

Klucz:

- S = 18
- Z = 25
- Y = 24
- F = 5
- R = 17

Zapiszmy to w tabeli:

Pozycja	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Litera jawna	K	O	M	P	U	T	E	R
Numer jawny (P_i)	10	14	12	15	20	19	4	17
Litera klucza	S	Z	Y	F	R	S	Z	Y
Numer klucza (K_i)	18	25	24	5	17	18	25	24

3.3. Szyfrowanie: $C_i = (P_i + K_i) \bmod 26$

Liczymy dla każdej pozycji:

1. **Pozycja 1**
    - $P_1 = 10$ (K), $K_1 = 18$ (S)
    - $C_1 = (10 + 18) \bmod 26 = 28 \bmod 26 = 2$
    - 2 → litera **C**
2. **Pozycja 2**
    - $P_2 = 14$ (O), $K_2 = 25$ (Z)
    - $C_2 = (14 + 25) \bmod 26 = 39 \bmod 26 = 13$
    - 13 → litera **N**
3. **Pozycja 3**
    - $P_3 = 12$ (M), $K_3 = 24$ (Y)
    - $C_3 = (12 + 24) \bmod 26 = 36 \bmod 26 = 10$
    - 10 → litera **K**
4. **Pozycja 4**
    - $P_4 = 15$ (P), $K_4 = 5$ (F)
    - $C_4 = (15 + 5) \bmod 26 = 20$
    - 20 → litera **U**
5. **Pozycja 5**
    - $P_5 = 20$ (U), $K_5 = 17$ (R)
    - $C_5 = (20 + 17) \bmod 26 = 37 \bmod 26 = 11$
    - 11 → litera **L**
6. **Pozycja 6**
    - $P_6 = 19$ (T), $K_6 = 18$ (S)
    - $C_6 = (19 + 18) \bmod 26 = 37 \bmod 26 = 11$
    - 11 → litera **L**
7. **Pozycja 7**
    - $P_7 = 4$ (E), $K_7 = 25$ (Z)
    - $C_7 = (4 + 25) \bmod 26 = 29 \bmod 26 = 3$
    - 3 → litera **D**

8. **Pozycja 8**
    - P_8 = 17 (R), K_8 = 24 (Y)
    - C_8 = (17 + 24) mod 26 = 41 mod 26 = 15
    - 15 → litera **P**

Zbierzmy wyniki:

Pozycja	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
P_i (jawny)	10	14	12	15	20	19	4	17
K_i (klucz)	18	25	24	5	17	18	25	24
$C_i = (P_i+K_i) \bmod 26$	2	13	10	20	11	11	3	15
Litera szyfru	C	N	K	U	L	L	D	P

Ostateczny szyfrogram:

CNKULLDP

3.4. Deszyfrowanie: $P_i = (C_i - K_i) \bmod 26$

Sprawdzamy, czy odzyskamy KOMPUTER.

Weźmy szyfrogram CNKULLDP i ten sam, powtarzający się klucz SZYFRSZY.

1. Zamiana szyfrogramu na numery:

- C = 2
- N = 13
- K = 10
- U = 20
- L = 11
- L = 11
- D = 3
- P = 15

2. Klucz w numerach (jak wcześniej):

- S = 18
- Z = 25
- Y = 24
- F = 5
- R = 17
- S = 18
- Z = 25
- Y = 24

Tabela:

Pozycja	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Litera szyfrogramu	C	N	K	U	L	L	D	P
C_i	2	13	10	20	11	11	3	15
Litera klucza	S	Z	Y	F	R	S	Z	Y
K_i	18	25	24	5	17	18	25	24

Teraz obliczamy:

- $P_i = (C_i - K_i) \bmod 26$

1. Pozycja 1:
    - $P_1 = (2 - 18) \bmod 26 = (-16) \bmod 26 = 10 \rightarrow$ **K**
2. Pozycja 2:
    - $P_2 = (13 - 25) \bmod 26 = (-12) \bmod 26 = 14 \rightarrow$ **O**
3. Pozycja 3:
    - $P_3 = (10 - 24) \bmod 26 = (-14) \bmod 26 = 12 \rightarrow$ **M**
4. Pozycja 4:
    - $P_4 = (20 - 5) \bmod 26 = 15 \rightarrow$ **P**
5. Pozycja 5:
    - $P_5 = (11 - 17) \bmod 26 = (-6) \bmod 26 = 20 \rightarrow$ **U**
6. Pozycja 6:
    - $P_6 = (11 - 18) \bmod 26 = (-7) \bmod 26 = 19 \rightarrow$ **T**
7. Pozycja 7:
    - $P_7 = (3 - 25) \bmod 26 = (-22) \bmod 26 = 4 \rightarrow$ **E**
8. Pozycja 8:
    - $P_8 = (15 - 24) \bmod 26 = (-9) \bmod 26 = 17 \rightarrow$ **R**

Otrzymujemy:

KOMPUTER

Tekst jawny został poprawnie odzyskany, więc szyfrowanie i deszyfrowanie wykonano poprawnie.